

《算法设计技巧与分析》

第9章 NP完全问题

朱友文

zhuyw@nuaa.edu.cn

易解问题与难解问题

- 如果对于一个问题 Π 存在一个算法，时间复杂性为 $O(n^k)$ ，其中 n 是问题规模， k 是一个非负整数，则称问题 Π 存在**多项式时间**算法。这类算法在**可以接受的时间内**实现问题求解，比如：排序、串匹配、矩阵相乘。
- 现实世界里还存在很多问题至今没有找到多项式时间算法，计算时间是指数和超指数函数（如 2^n 和 $n!$ ），随着问题规模的增长而**快速增长**
- 通常将前者看作是**易解问题**，后者看作**难解问题**

易解问题与难解问题的主要区别

在学术界已达成这样的共识：把多项式时间复杂性作为易解问题与难解问题的分界线，主要原因有：

1) 多项式函数与指数函数的增长率有本质差别

问题规模 n	多项式函数					指数函数	
	logn	n	nlogn	n²	n³	2ⁿ	n!
1	0	1	0.0	1	1	2	1
10	3.3	10	33.2	100	1000	1024	3628800
20	4.3	20	86.4	400	8000	1048376	2.4E18
50	5.6	50	282.2	2500	125000	1.0E15	3.0E64
100	6.6	100	664.4	10000	1000000	1.3E30	9.3E157

2) 计算机性能的提高对易解问题与难解问题算法的影响

假设求解同一个问题有5个算法A1~A5，时间复杂度T(n)如下表，假定计算机C2的速度是计算机C1的10倍。下表给出了在相同时间内不同算法能处理的问题规模情况：

T(n)	n (C1)	n' (C2)	变化	n'/n
10n	1000	10000	$n'=10n$	10
20n	500	5000	$n'=10n$	10
5nlogn	250	1842	$\sqrt{10} \ n < n' < 10n$	7.37
2n ²	70	223	$\sqrt{10} \ n$	3.16
2 ⁿ	13	16	$n'=n+\log 10 \approx n+3$	≈ 1

3) 多项式时间复杂性忽略了系数，不影响易解问题与难解问题的划分

问题规模n	多项式函数			指数函数	
	n^8	$10^8 n$	n^{1000}	1.1^n	$2^{0.01n}$
5	390625	5×10^8	5^{1000}	1.611	1.035
10	10^8	10^9	10^{1000}	2.594	1.072
100	10^{16}	10^{10}	10^{2000}	13780.6	2
1000	10^{24}	10^{11}	10^{3000}	2.47×10^{41}	1024

观察结论： $n \leq 100$ 时，（不自然的）多项式函数值大于指数函数值，但 n 充分大时，指数函数仍然超过多项式函数。

9.2 P类问题和NP类问题

这个划分标准是基于对所谓判定问题的求解方式。

先看看什么是判定问题。事实上，实际应用中的大部分问题可以很容易转化为相应的判定问题，如：

- 数组中任意两个元素差值绝对值的最小值 $\Rightarrow$ 给定 k ，是否存在两个元素差值的绝对值小于 k ？
- 图着色问题：给定无向连通图 $G=(V,E)$ ，求最小色数 k ，使得任意相邻顶点具有不同的着色 $\Rightarrow$
给定无向连通图 $G=(V,E)$ 和正整数 k ，是否可以用 k 种颜色.....？

确定性算法与P类问题

对判定问题求解，可以采用确定性算法

- 定义1(确定性算法): 设A是求解问题 Π 的一个算法，如果在算法的整个执行过程中，每一步只有一个确定的选择，则称算法A是确定性算法。
 - 特点：对同一输入实例，运行算法A，所得结果是一样的。
- 定义2(P类问题): 如果对于某个判定问题 Π ，存在一个非负整数k，对于输入规模为n的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到yes或no的答案，则称该判定问题 Π 是一个P(Polynomial)类问题。
 - 事实上，所有易解问题都是P类问题。

非确定性算法与NP类问题

- 定义3(非确定性算法): 设A是求解问题 Π 的一个算法, 如果算法A以如下**猜测+验证的方式**工作, 称算法A为非确定性(nondeterminism)算法:
- **猜测阶段**: 对问题的输入实例产生一个任意字符串y, 在算法的每次运行, y可能不同, 因此猜测是以非确定的形式工作。这个工作一般可以在线性时间内完成。
- **验证阶段**: 在这个阶段, 用一个确定性算法验证两件事: 首先验证猜测的y是否是合适的形式, 若不是, 则算法停下并回答no; 若是合适形式, 则继续检查它是否是问题x的解, 如果确实是x的解, 则停下并回答yes, 否则停下并回答no。要求验证阶段在多项式时间内完成。

注意对非确定性算法输出**yes/no**的理解:

- 若输出**no**，并不意味着不存在一个满足要求的解，因为猜测可能不正确；若输出**yes**，则意味着对于该判定问题的某一输入实例，至少存在一个满足要求的解。

NP类问题

- 定义4(**NP类问题**): 如果对于判定问题 Π , 存在一个非负整数 k , 对于输入规模为 n 的实例, 能够以 $O(n^k)$ 的时间运行一个**非确定性算法**, 得到**yes/no**的答案, 则该判断问题 Π 是一个**NP(nondeterministic polynomial)**类问题。

※注意: **NP**类问题是对于判定问题定义的, 事实上, 可以在多项式时间内应用非确定性算法解决的所有问题都属于**NP**类问题。

通俗地说: 多项式时间内可以**验证**某个值是否是正确答案

关于P与NP关系的初步思考

--从字面含义

- 1) 若问题 Π 属于P类，则存在一个多项式时间的确定性算法，对它进行判定或求解；显然，也可以构造一个多项式时间的非确定性算法，来验证解的正确性，因此，问题也属NP类。因此，显然有 $P \subseteq NP$
- 2) 若问题 Π 属于NP类，则存在一个多项式时间的非确定性算法，来猜测并验证它的解；但不一定能构造一个多项式时间的确定性算法，来对它进行求解或判定。

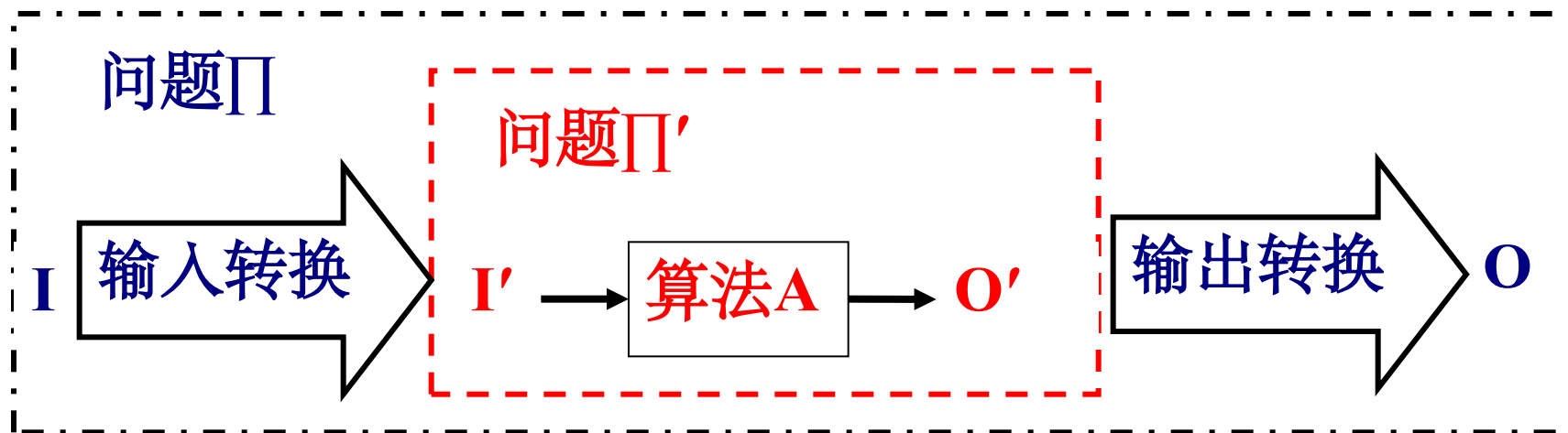
➤ 因此，人们猜测 $P \neq NP$ ，但是否成立，至今未得到证明

9.3 NP完全问题

- NP完全问题是NP类问题的子类，一个具有特殊性质与特殊意义的子类。

问题归约

- **NP**类问题在最坏情况下的时间复杂性一般都是快速增长的指数函数。我们希望能够**在NP类问题内部**找到一种方法，比较两个问题的复杂性。



多项式时间归约

- 定义5(多项式时间归约):若在 $O(\tau(n))$ 时间内完成上述输入/输出转换, 则称问题 Π 以 $\tau(n)$ 时间归约到问题 Π' , 记为

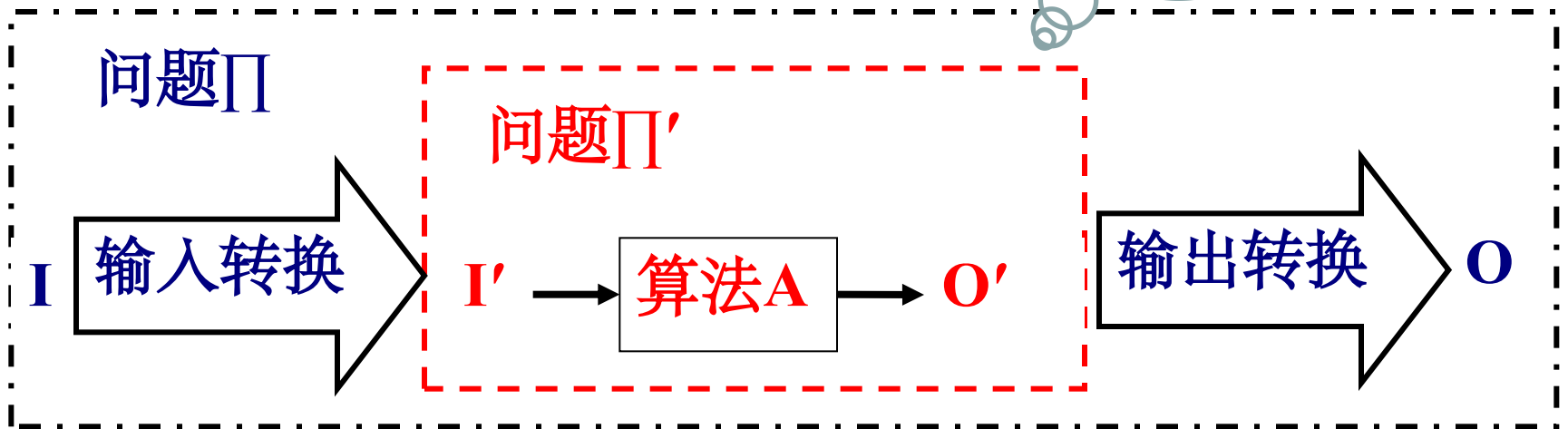
$$\Pi \propto_{\tau(n)} \Pi'$$

- 其中, n 为问题规模; 若在多项式时间内完成上述转换, 则称问题 Π 以多项式时间归约到问题 Π' , 记为 $\Pi \propto_{\text{poly}} \Pi'$

多项式时间归约

- $\Pi \propto_{\text{poly}} \Pi'$

可以多次调用
算法A，但是调
用次数是多项
式次



举例：多项式时间归约

排序问题的算法A，输入为 $x_1, x_2, \dots, x_n$ ，输出为非降序序列

$$x_{i1} \leq x_{i2} \leq \dots \leq x_{in} ;$$

配对问题II：输入两组数 $X=(x_1, x_2, \dots, x_n)$ 和 $Y=(y_1, y_2, \dots, y_n)$ ，输出是两组元素的非降序依次配对。

求解配对问题，需要进行三个变换

- 将配对问题的输入 X, Y 变成排序问题的两个输入 I_1, I_2 ；
- 应用算法A对 I_1, I_2 分别排序，得到两个排序输出 O_1, O_2 ；
- 将两个排序输出 O_1, O_2 转换成配对问题的输出 O 。

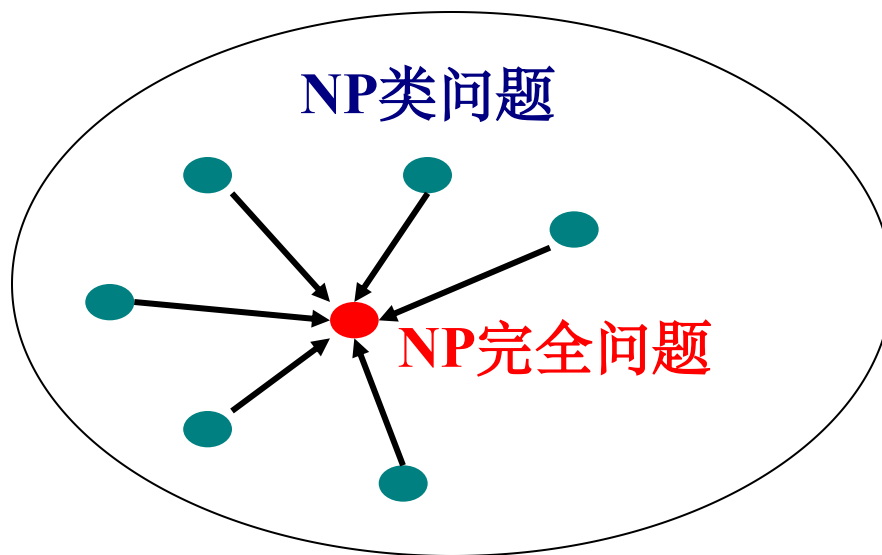
这些操作可以在多项式时间内完成。

多项式时间归约的性质

- **传递性**: 设 Π 、 Π' 和 Π'' 是3个判定问题, 若 $\Pi \propto_{\text{poly}} \Pi'$, 且 $\Pi' \propto_{\text{poly}} \Pi''$, 则 $\Pi \propto_{\text{poly}} \Pi''$ 。
- **自反性**: $\Pi \propto_{\tau(n)} \Pi$

NP完全问题

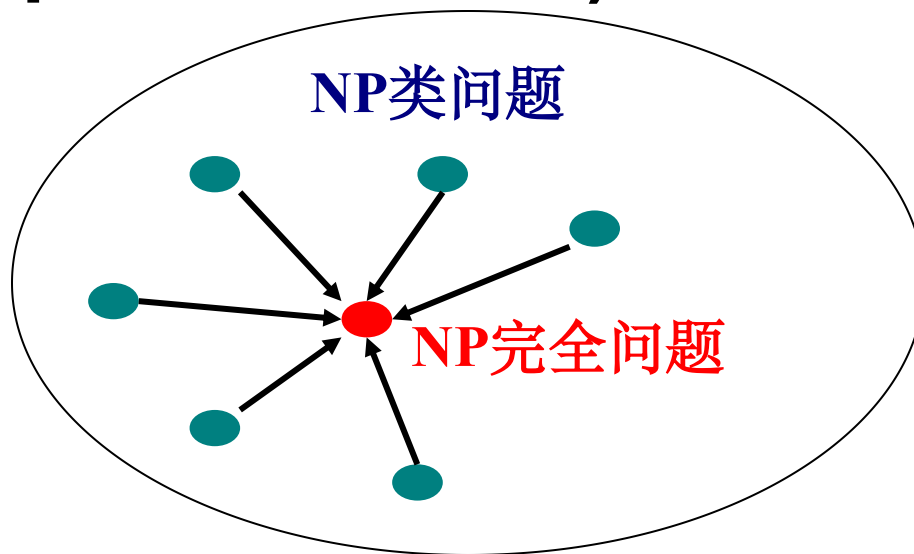
NP完全问题是一类具备如下特殊性质的NP类问题



- Π (该问题本身) 就是一个NP类问题
- 每个NP类问题都可以多项式时间归约为 Π

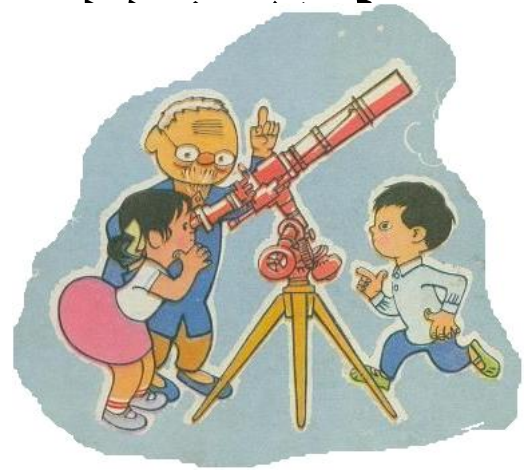
NP完全问题的定义

- 定义6(**NP完全问题**): 令 Π 是一个判定问题, 如果问题 Π 属于NP类问题, 并且对NP类问题中的每一个问题 Π' , 都有 $\Pi' \leq_{\text{poly}} \Pi$, 则称判定问题 Π 是一个NP完全问题(NP complete problem, NPC)。

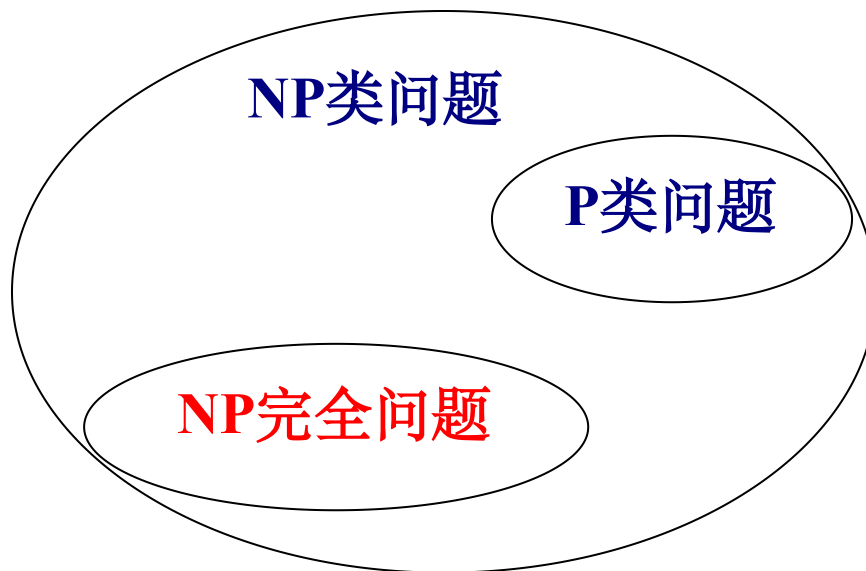


对“NP完全问题”

- **NP完全问题是NP类问题中最难的一类问题**，至今已经发现了几千个，但一个也没有找到多项式时间算法。
- 如果某一个**NP完全问题**能在多项式时间内解决，则每一个**NP完全问题**都能在多项式时间内解决。
- **这些问题也许存在多项式时间算法**，因为计算机科学是相对新生的科学，肯定还有新的算法设计技术有待发现；
- **这些问题也许不存在多项式时间算法**，但目前缺乏足够的依据来证明这一点。



P类问题、NP类问题、NP完全问题的关系

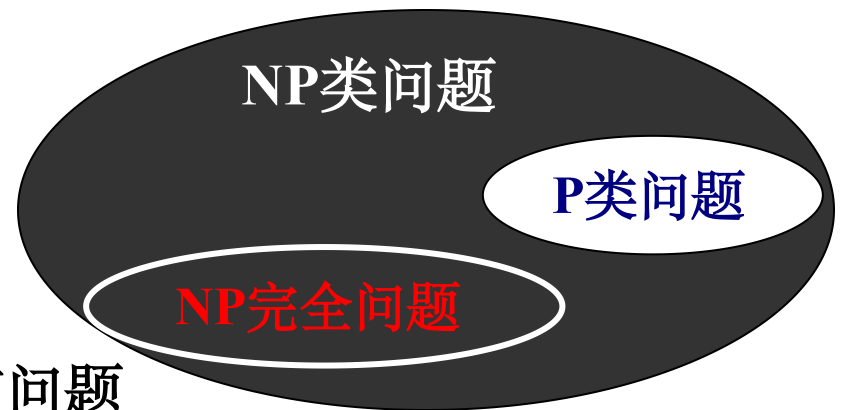


关于P与NP关系的再思考

--从深层意义

至今没有人能证明是否 $P \neq NP$ 。

- 若 $P=NP$ ，说明NP类中所有问题（包括NP完全问题）都具有多项式时间算法；
- 若 $P \neq NP$ ，说明NP类中除P类之外的所有问题（包括NP完全问题）都不存在多项式时间算法。

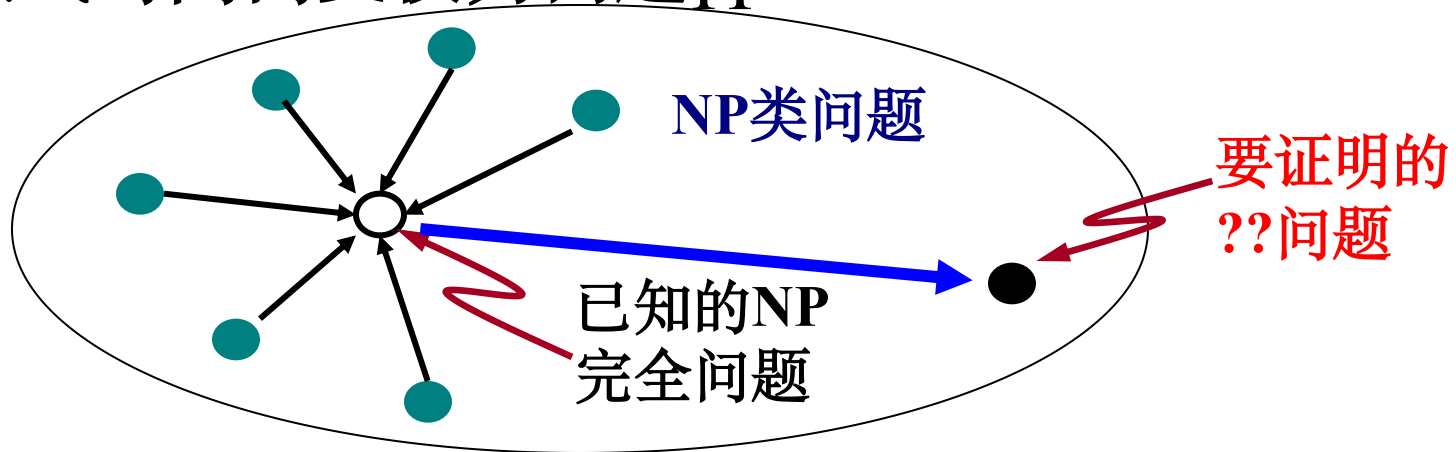


无论哪种答案，都将为算法设计提供重要的指导和依据。

目前人们普遍猜测： $P \neq NP$

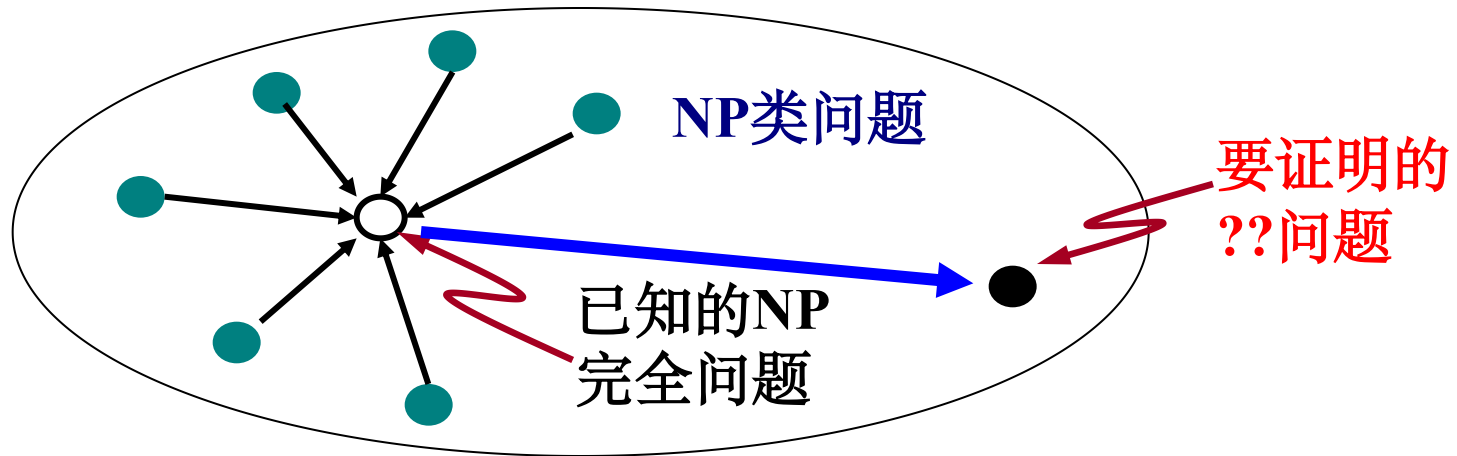
如何证明一个问题是NP完全的

- 根据NP完全问题的定义，需要分两个步骤：
 - 证明问题 Π 是NP类问题；即可以在多项式时间内以确定性算法验证一个任意生成的串，以确定它是否为问题的一个解
 - 证明NP类问题中的每一个问题都能以多项式时间归约为问题 Π 。由于多项式问题变换具有传递性，所以，只需证明一个已知的NP完全问题能够在多项式时间内变换为问题 Π 。



如何证明一个问题是NP完全的

万事开头难：第一个NP完全问题怎么证明？



最基本的NP完全问题

- 1971年，Cook证明了Cook定理：布尔表达式的可满足性(SAT)问题是**NP完全的**

可满足性问题即合取范式的可满足性问题，来源于许多实际的逻辑推理的应用。合取范式形如 $A_1 \wedge A_2 \wedge \dots \wedge A_n$ ，其中子句 A_i ($1 \leq i \leq n$)形如： $a_1 \vee a_2 \vee \dots \vee a_k$ ，其中 a_j 称为文字，为布尔变量。SAT问题是指：是否存在一组对所有布尔变量的赋值，使得整个合取范式为真？例如

$$f = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3 \vee x_4 \vee \bar{x}_5) \wedge (x_1 \vee \bar{x}_3 \vee x_4)$$

当 x_1 和 x_3 都为真、其余文字任意赋值时， f 值为真。

其他NP完全问题

- 可满足性(SAT)问题是第一个被证明的NP完全问题。1972年, Karp证明了十几个问题都是NP完全的。这些NP完全问题的证明思想和技巧, 以及利用他们证明的几千个NP完全问题, 极大地丰富了NP完全理论。

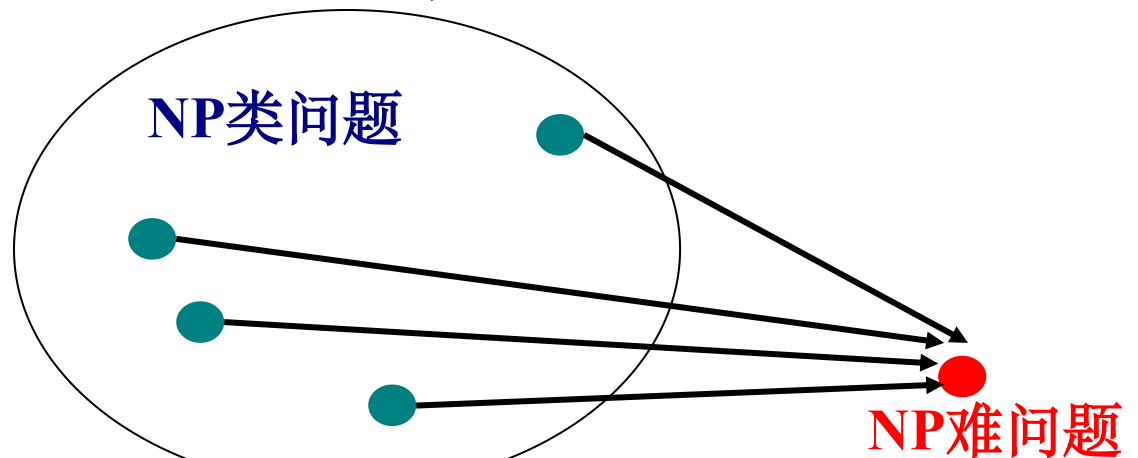
- 已证明的NP完全问题:

SAT问题、最大团问题、图着色问题、哈密顿回路问题、旅行商问题、背包问题、最长路径问题、扫雷游戏...



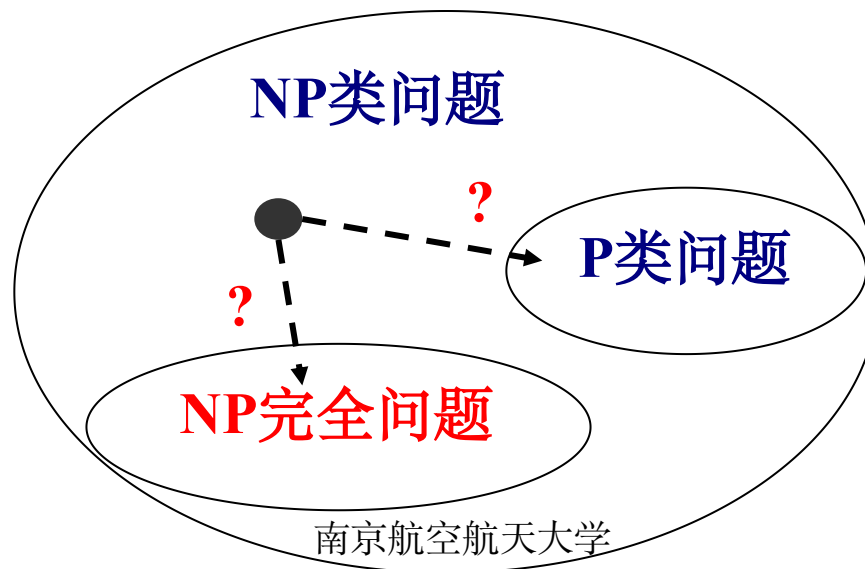
NP难问题

难解问题中还有一类问题，虽然也能证明所有的NP类问题可以以多项式时间归约到问题 Π ，**但并不能证明 Π 也是NP类问题**，所以 Π 不是NP完全的。但问题 Π 至少与任意NP类问题有同样的难度，这样的问题称为NP难问题



NPI类问题

- **NP**类问题中还有一些问题，人们不知道是属于**P**类还是属于**NP**完全问题，还有**待于证明其归属**
- 这些问题是**NP**完全问题的可能性非常小，也因为不相信他们在**P**中，我们**人为地增加另一问题类**来接纳这类问题，这个类称为**NPI(NP-Intermediate)类**。



关于NPI类问题的评述

- **NPI类是一个人为定义的、动态的概念**，随着人们对问题研究的深入，许多**NPI类问题**逐渐被明白无误地证明他们原本属于**P类问题**或**NP完全问题**。
- 例如：线性规划问题、素数判定问题等，在二者没有被证明他们均属于**P类问题**之前，人们一直将他们归于**NPI类问题**

一个例子

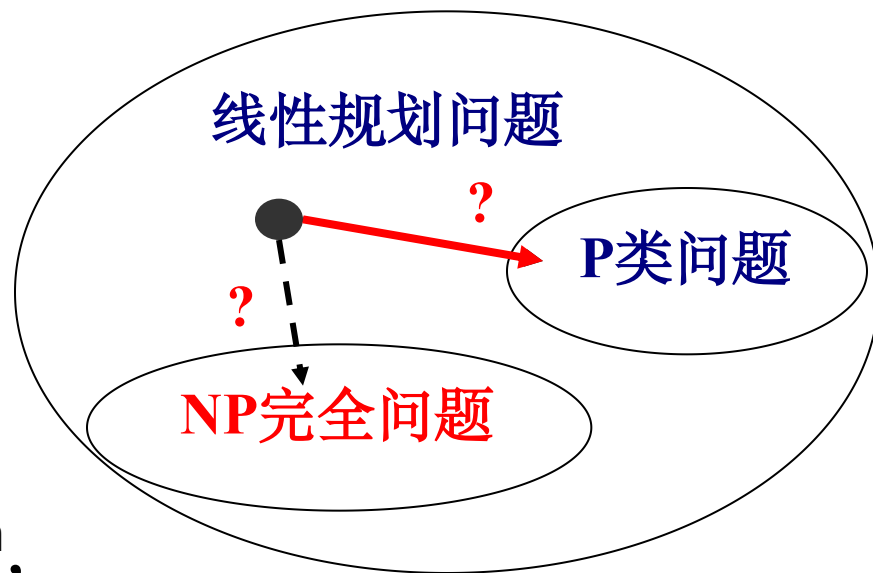
- 线性规划问题

设 $A \in R^{m \times n}$, $x \in R^n$, $b \in R^m$,

在满足 $Ax=b$, $x \geq 0$ 约束下,

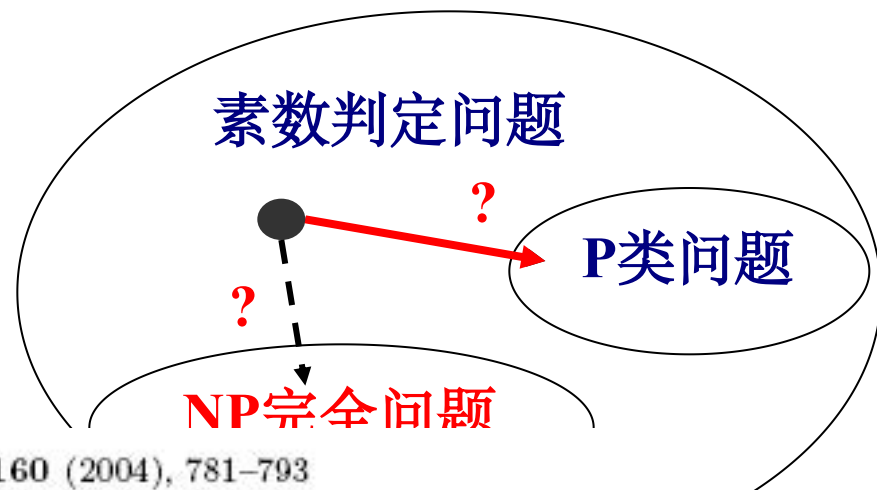
使目标函数 $c^T x$ 达到最大值, 其中 $c \in R^n$.

- 长期以来, 线性规划问题没有多项式时间解法, 也无法证明它是NP完全问题。直到20世纪80年代, 这个问题得到解决, 发现了多项式时间算法



另一个例子

素数判定问题



Annals of Mathematics, 160 (2004), 781–793

- 结
- 绍
- 绍
- A
- S
- 算

PRIMES is in P

By MANINDRA AGRAWAL, NEERAJ KAYAL, and NITIN SAXENA*

Abstract

We present an unconditional deterministic polynomial-time algorithm that determines whether an input number is prime or composite.

NP完全问题的处理技术简介

NP完全问题是计算机难以处理的，但是实际中经常遇到，我们无法回避这些问题。因此，人们提出了解决**NP完全问题**的各种方法：

- **采用先进的算法设计技术**

- 问题规模不是很大时，采用动态规划法、回溯法、分枝限界法等。

- **近似算法、贪心算法**

- 很多问题的解允许有一定的误差，只要给出的解是合理的、可接受的。

NP完全问题的处理技术简介

- 随机算法

- 例如随机采样算法，穷举+挑一 vs. 随机化采样， $\Theta(n)$
- 或以较小的错误概率为代价，获得计算时间的大幅减少。

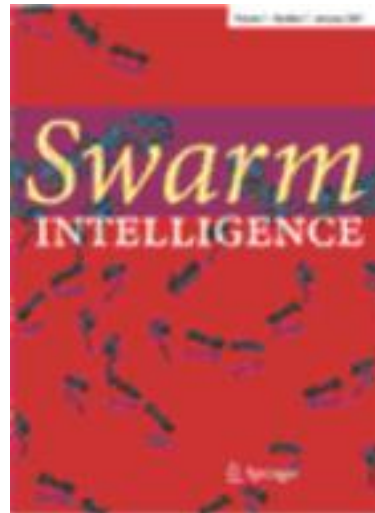
- 并行计算

- 利用多台CPU共同完成一项计算，虽然从原理上说，增强计算性能并不能从根本上解决NP完全问题，但这也是解决NP完全问题的措施之一。近年来的成就如129位(bit)大整数的分解、“深蓝”下棋程序等。

NP完全问题的处理技术简介

- 智能算法

- 遗传算法
- 人工神经网络
- 模拟退火算法
- 禁忌搜索算法
- DNA计算
- 人工免疫算法
- 蚁群算法



Swarm Intelligence: a new journal dedicated to reporting on developments in the discipline of swarm intelligence. Published by Springer. **Editor-in-Chief: Marco Dorigo.**

(蚁群算法小游戏) <http://www.atlas-zone.com/complex/temp/ant/ant.htm>